

Project Checkpoint 2:

James Hawkins, Enyu Ling, Akhil Shastry, Jason Yu

GitHub Link (has .ipynb file): https://github.com/jasonyu8379/CMSC320_Semester_Project/tree/main
(https://github.com/jasonyu8379/CMSC320_Semester_Project/tree/main)

Conclusion 1: Race versus Presidential Candidate Choice (2024)

In this section, we will parse through the American National Election Survey Dataset and examine the cross-categorical relationship between a voter's racial group and their preferred candidate in the 2024 Presidential Election.

```
In [2]: # Import Libraries
import pandas as pd
import scipy as sp
import numpy as np
from matplotlib import pyplot
```

```
In [3]: # Loading in the dataset

anes2024_csv = "../../../dataset/data_2024/anes_timeseries_2024_csv_20250808.csv"
df2024 = pd.read_csv(anes2024_csv)

# Select relevant variables (per the Codebook)
df2024_select = df2024[["V241501x", "V241567x", "V241566x", "V241039", "V240108a",
                        "V240002c", "V242067"]]

df2024_select
```

C:\Users\james\AppData\Local\Temp\ipykernel_9764\1563674870.py:4: DtypeWarning: Columns (0: V240101a, 1: V240101c, 2: V240101d, 3: V240103a, 4: V240103c, 5: V240103d, 6: V240104a, 7: V240104c, 8: V240104d, 9: V240105a, 10: V240105c, 11: V240105d, 12: V240106a, 13: V240106c, 14: V240106d, 15: V240108a, 16: V240108c, 17: V240108d, 18: V243002, 19: V243003, 20: V243004, 21: V243005, 22: V243006, 23: V243009, 24: V243050, 25: V243051, 26: V243052, 27: V243053, 28: V243056, 29: V243057, 30: V243058) have mixed types. Specify dtype option on import or set low_memory=False.

```
df2024 = pd.read_csv(anes2024_csv)
```

Out[3]:

	V241501x	V241567x	V241566x	V241039	V240108a	V240002c	V242067
0	3	5	27	-1	0.809064	2	2
1	4	5	26	-1	2.575472	2	-1
2	1	5	24	-1	0.798864	2	1
3	4	3	12	-1	0.244757	2	-1
4	1	3	15	-1	0.239209	2	2
...	...	...	...	...	...	...	...
5516	-8	2	5	-1	0.19183	1	-6
5517	1	3	16	-1	2.869063	2	1
5518	2	3	12	-9	2.092296	1	-6
5519	2	4	19	-1	2.826738	2	1
5520	1	4	22	-1	1.309934	2	2

5521 rows × 7 columns

In [4]: *# Dictionaries for conversion*

```
race_dict = { # V241501x
    -9: "Refused",
    -8: "Don't Know",
    -4: "Error",
    1: "White", #"White, non-Hispanic",
    2: "Black", #"Black, non-Hispanic",
    3: "Hispanic",
    4: "Asian/Pacific-Islander", #"Asian or Native Hawaiian/other Pacific Islander, non-Hispanic",
    5: "Native American/Other", #"Native American/Alaska Native or other race, non-Hispanic",
    6: "Multiple Races", #"Multiple races, non-Hispanic",
}

president_dict = { # V242067
    -9: "Refused",
    -1: "Inapplicable",
    1: "Harris", #"Kamala Harris",
    2: "Trump", #"Donald Trump",
    3: "RFK Jr.", #"Robert F. Kennedy, Jr.",
    4: "West", #"Cornel West",
    5: "Stein", #"Jill Stein",
    6: "Other", #"Another candidate {SPECIFY}"
}

complete_survey_dict = {
    1: 0,
    2: 1
}
```

```
In [5]: # Making the dataset more readable
df2024_select["Race"] = df2024_select["V241501x"].map(race_dict)
df2024_select["President Vote"] = df2024_select["V242067"].map(president_dict)
df2024_select["Completed Survey"] = df2024_select["V240002c"].map(complete_survey_dict)

# Clean up 'Weights': replace empty strings with 0.
df2024_select["Weights"] = df2024_select["V240108a"]
df2024_select["Weights"] = df2024_select["Weights"].apply(lambda x: 0 if x == '' or x == ' ' else x)
df2024_select["Weights"] = df2024_select["Weights"].astype(float)

df2024_clean = df2024_select[["Race", "President Vote",
                              "Completed Survey", "Weights"]]

df2024_clean
```

Out[5]:

	Race	President Vote	Completed Survey	Weights
0	Hispanic	Trump	1	0.809064
1	Asian/Pacific-Islander	Inapplicable	1	2.575472
2	White	Harris	1	0.798864
3	Asian/Pacific-Islander	Inapplicable	1	0.244757
4	White	Trump	1	0.239209
...	...	...	...	...
5516	Don't Know	NaN	0	0.191830
5517	White	Harris	1	2.869063
5518	Black	NaN	0	2.092296
5519	Black	Harris	1	2.826738
5520	White	Trump	1	1.309934

5521 rows × 4 columns

```
In [6]: # Dealing with missing values
df2024_clean.dropna()
df2024_clean

# Get only those who completed the post-election survey
df2024_relevant = df2024_clean[df2024_clean["Completed Survey"] == 1]

df2024_relevant
```

Out[6]:

	Race	President Vote	Completed Survey	Weights
0	Hispanic	Trump	1	0.809064
1	Asian/Pacific-Islander	Inapplicable	1	2.575472
2	White	Harris	1	0.798864
3	Asian/Pacific-Islander	Inapplicable	1	0.244757
4	White	Trump	1	0.239209
...	...	...	...	...
5514	White	Inapplicable	1	0.797417
5515	Hispanic	Harris	1	0.129755
5517	White	Harris	1	2.869063
5519	Black	Harris	1	2.826738
5520	White	Trump	1	1.309934

4964 rows × 4 columns

```
In [7]: # EDA
# Cross-Tab of Race and President Choice
president_race_tab = pd.crosstab(index = df2024_relevant["Race"],
                                columns = df2024_relevant["President Vote"],
                                values = df2024_relevant["Weights"],
                                aggfunc=sum, normalize="index").fillna(0)

president_race_tab_percents = president_race_tab.round(4) * 100
president_race_tab_percents
```

Out[7]:

	President Vote	Harris	Inapplicable	Other	Refused	Stein	Trump	West
Race								
Asian/Pacific-Islander		47.93	22.88	1.34	0.00	0.22	27.35	0.27
Black		62.35	29.17	1.13	0.66	0.08	6.18	0.42
Hispanic		39.23	39.40	0.66	0.45	1.55	18.52	0.19
Multiple Races		34.58	32.40	2.78	0.29	0.62	28.70	0.63
Native American/Other		23.54	44.32	4.19	0.00	0.00	27.94	0.00
Refused		20.93	25.04	0.00	9.08	0.00	44.24	0.71
White		35.47	22.56	1.45	1.15	0.49	38.78	0.09

```
In [8]: # Simplifying Data to only people that answered with an actual category.
df2024_simplified = df2024_relevant[df2024_relevant["Race"] != "Refused"]

# Turn third-party candidates into one category.
def collapse_third_party(vote):
    if vote in ["West", "Stein", "Other"]:
        return "Other"
    else:
        return vote

df2024_simplified["President Vote"] = df2024_simplified["President Vote"].apply(collapse_third_party)

df2024_simplified = df2024_simplified[df2024_simplified["President Vote"] != "Refused"]
df2024_simplified = df2024_simplified[df2024_simplified["President Vote"] != "Inapplicable"]

df2024_simplified
```

Out[8]:

	Race	President Vote	Completed Survey	Weights
0	Hispanic	Trump	1	0.809064
2	White	Harris	1	0.798864
4	White	Trump	1	0.239209
6	White	Trump	1	0.290135
10	White	Harris	1	0.940042
...	...	...	...	...
5507	White	Other	1	3.652063
5515	Hispanic	Harris	1	0.129755
5517	White	Harris	1	2.869063
5519	Black	Harris	1	2.826738
5520	White	Trump	1	1.309934

3663 rows × 4 columns

```
In [9]: # Cross-Tab with simplified, positive voter dataset
race_president_cross_tab = pd.crosstab(index = df2024_simplified["Race"],
                                         columns = df2024_simplified["President Vote"],
                                         values = df2024_simplified["Weights"],
                                         aggfunc=sum, normalize="index").fillna(0)

race_president_cross_tab_percents = race_president_cross_tab.round(4) * 100
race_president_cross_tab_percents
```

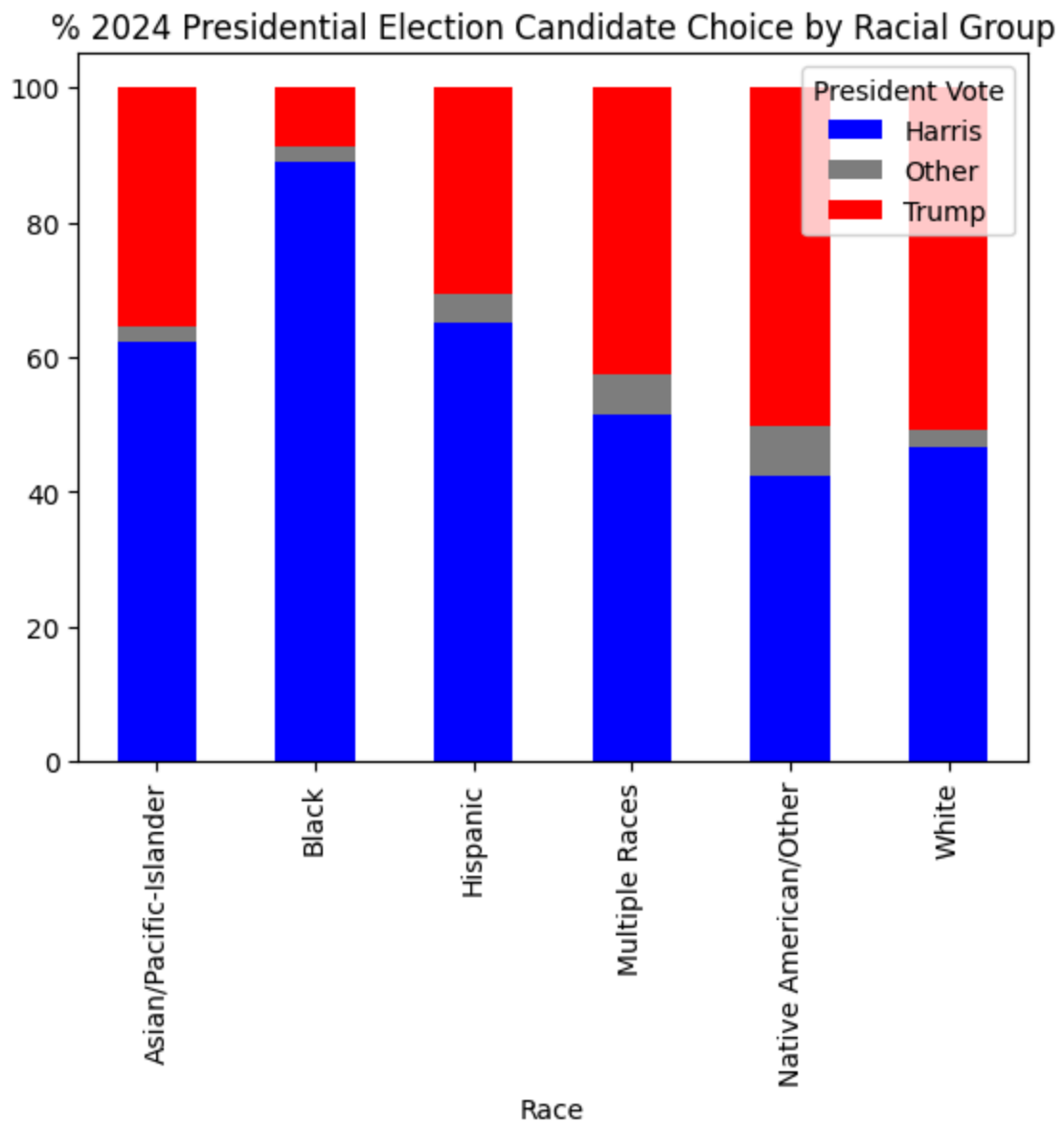
Out[9]:

	President Vote	Harris	Other	Trump
Race				
Asian/Pacific-Islander		62.15	2.38	35.47
Black		88.87	2.33	8.80
Hispanic		65.23	3.99	30.79
Multiple Races		51.37	5.99	42.64
Native American/Other		42.28	7.53	50.19
White		46.50	2.67	50.83

```
In [10]: # Bar plot
race_v_president_barplot = race_president_cross_tab_percents.plot.bar(
    stacked = True,
    title = "% 2024 Presidential Election Candidate Choice by Racial Group",
    color = ['blue', 'grey', 'red']
)

race_v_president_barplot
```

```
Out[10]: <Axes: title={'center': '% 2024 Presidential Election Candidate Choice by Racial Group'}, xlabel='Race'>
```



Based on this bar chart and the cross-tab, it appears that there is a difference in candidate preference across the different racial groups. Let us test this with a hypothesis:

Null Hypothesis: there is no difference in candidate preference (for the 2024 Presidential Election) across different racial groups.

Alternative Hypothesis: there is a difference in candidate preference across racial groups.

We will use a Chi² test to see if the above results are likely to have arisen given random chance.

```
In [11]: # Chi^2 Test on this Cross-Tabulation

# We need to actually remake the cross-tab, because Chi^2 cares about counts.
race_president_cross_tab_count = pd.crosstab(index = df2024_simplified["Race"],
                                              columns = df2024_simplified["President Vote"],
                                              values = df2024_simplified["Weights"],
                                              aggfunc=sum).fillna(0)

race_president_test = sp.stats.chi2_contingency(race_president_cross_tab_count)
p_value = race_president_test.pvalue
print(f"P-Value: {float(p_value)}")

race_president_cross_tab_count
# The values in this cross-tab are floats due to weighting, but they still function as
# cross-category frequencies.
```

P-Value: 5.313565461524059e-54

Out[11]:

	President Vote	Harris	Other	Trump
Race				
Asian/Pacific-Islander		105.033297	4.014358	59.938848
Black		332.888678	8.738046	32.972940
Hispanic		256.988709	15.701025	121.302511
Multiple Races		74.991483	8.740201	62.245260
Native American/Other		4.740423	0.844645	5.627008
White		1060.584502	60.941264	1159.539760

Based on this Chi² test, we can conclude that there is a statistically significant difference between the frequencies of votes for Harris, Trump, and other candidates based on racial group of the voter. The p-value is well less than the statistical significance threshold of $\alpha = 0.05$

It should be noted that there are possible confounders to this relationship, namely income group and age.

Outliers here would be votes for other candidates besides Harris and Trump, which make up a substantial minority of voters. We filtered out non-voters and people who refused to or otherwise did not answer the questions about race and who they voted for in the Presidential Election. We also (necessarily) only considered survey entries that completed the post-election survey (to indicate who they voted for). Imbalance of demographic groups should be mitigated for by the row weights, and the ANES has assured that its samples are representative.

Based on the cross-tab, it would appear that Asians, Hispanics, and especially African Americans were more likely to vote for Harris than Trump. People of multiple races were more likely to vote for Harris than Trump, but by less of a significant margin. Whites and Native Americans were more likely to vote for Trump, but also by less of a significant margin. It should be noted that the Native American sample is small, and so is perhaps biased. Whites are well overrepresented in this dataset, with Blacks, Hispanics, and Asians being the next highest groups.

Conclusion 2: Presidential Candidate Choice (2024) versus Age

```
In [19]: import pandas as pd
import scipy as sp
import numpy as np
from matplotlib import pyplot as plt
```

The columns that I used below are, V241458x (age of voter on election day), V242067 (who the person voted for), V240002c (Pre/Post Interview Completion Status) to get data on those who filled out the information after the election, and V240108b (sampling weights per row) is needed to perform the statistical tests seen later on.

The Goal of this analysis is to understand the relationship between the age of the voter and their vote choice, so we can see if there is a correlation between the two continuous variables.

```
In [12]: anes2024_csv = "../../../dataset/data_2024/anes_timeseries_2024_csv_20250808.csv"
df = pd.read_csv(anes2024_csv)
df_useful = df[["V241458x", "V242067", "V240002c", "V240108b"]]
df_useful
```

C:\Users\james\AppData\Local\Temp\ipykernel_9764\3773667444.py:2: DtypeWarning: Columns (0: V240101a, 1: V240101c, 2: V240101d, 3: V240103a, 4: V240103c, 5: V240103d, 6: V240104a, 7: V240104c, 8: V240104d, 9: V240105a, 10: V240105c, 11: V240105d, 12: V240106a, 13: V240106c, 14: V240106d, 15: V240108a, 16: V240108c, 17: V240108d, 18: V243002, 19: V243003, 20: V243004, 21: V243005, 22: V243006, 23: V243009, 24: V243050, 25: V243051, 26: V243052, 27: V243053, 28: V243056, 29: V243057, 30: V243058) have mixed types. Specify dtype option on import or set low_memory=False.

```
df = pd.read_csv(anes2024_csv)
```

Out[12]:

	V241458x	V242067	V240002c	V240108b
0	50	2	2	.780974621163524
1	41	-1	2	2.50707498962934
2	44	1	2	.814400897948447
3	45	-1	2	.263490124186044
4	80	2	2	.216673241876399
...	...	...	...	...
5516	-2	-6	1	
5517	69	1	2	1.90730910362323
5518	-2	-6	1	
5519	28	1	2	2.56436072555069
5520	37	2	2	1.34915930327876

5521 rows × 4 columns

```
In [13]: # Changed the names of each of the columns

df_select = df[["V241458x", "V242067", "V240002c", "V240108b"]]

df_select = df_select.rename(columns={
    "V241458x": "Age",
    "V242067": "President Vote",
    "V240002c": "Completed Post Survey",
    "V240108b": "Weights"
})

df_select
```

Out[13]:

	Age	President Vote	Completed Post Survey	Weights
0	50	2	2	.780974621163524
1	41	-1	2	2.50707498962934
2	44	1	2	.814400897948447
3	45	-1	2	.263490124186044
4	80	2	2	.216673241876399
...	...	...	...	...
5516	-2	-6	1	
5517	69	1	2	1.90730910362323
5518	-2	-6	1	
5519	28	1	2	2.56436072555069
5520	37	2	2	1.34915930327876

5521 rows × 4 columns

```

In [14]: # changed the numbers that represent each "President Vote" to the name of said
         president

president_dict = { # V242067
    -9: "Refused",
    -1: "Inapplicable",
    1: "Harris", # "Kamala Harris",
    2: "Trump", # "Donald Trump",
    3: "RFK Jr.", # "Robert F. Kennedy, Jr.",
    4: "West", # "Cornel West",
    5: "Stein", # "Jill Stein",
    6: "Other", # "Another candidate {SPECIFY}"
}
df_select["President Vote"] = df["V242067"].map(president_dict)

# empty weight values represented as 0
df_select["Weights"] = df_select["Weights"].apply(lambda x: 0 if x == ' ' or x
== '' else x)
df_select["Weights"] = df_select["Weights"].astype(float)

# remove ages below 18 years old
df_select = df_select[df_select["Age"] >= 18]

df_select

```

Out[14]:

	Age	President Vote	Completed Post Survey	Weights
0	50	Trump	2	0.780975
1	41	Inapplicable	2	2.507075
2	44	Harris	2	0.814401
3	45	Inapplicable	2	0.263490
4	80	Trump	2	0.216673
...	...	...	...	...
5514	42	Inapplicable	2	0.596674
5515	80	Harris	2	0.094869
5517	69	Harris	2	1.907309
5519	28	Harris	2	2.564361
5520	37	Trump	2	1.349159

5242 rows × 4 columns

```
In [15]: # drop missing values
df_select = df_select.dropna()

# only keep data points where people finished the post survey
df_select = df_select[df_select["Completed Post Survey"] == 2]

# remove inapplicable or refused rows of people who didn't pick a candidate
df_select = df_select[df_select["President Vote"].isin(["Harris", "Trump", "We
st", "Stein", "Other"])]

df_select
```

Out[15]:

	Age	President Vote	Completed Post Survey	Weights
0	50	Trump	2	0.780975
2	44	Harris	2	0.814401
4	80	Trump	2	0.216673
6	41	Trump	2	0.292916
10	59	Harris	2	0.964122
...	...	...	...	...
5507	45	Other	2	3.252915
5515	80	Harris	2	0.094869
5517	69	Harris	2	1.907309
5519	28	Harris	2	2.564361
5520	37	Trump	2	1.349159

3531 rows × 4 columns

EDA

First we examine the basic characteristics of the age variable across the dataset, by looking at the mean, median, standard deviation, and distribution for each candidate.

```
In [16]: print("Age Data Statistics: ")
print("-----")
print(df_select["Age"].describe())
```

```
Age Data Statistics:
-----
count    3531.000000
mean      55.174172
std       16.769812
min       18.000000
25%       41.000000
50%       57.000000
75%       69.000000
max       80.000000
Name: Age, dtype: float64
```

```
In [17]: print("Mean Age by Candidate: ")
print("-----")
print(df_select.groupby("President Vote")["Age"].mean())
```

```
Mean Age by Candidate:
-----
President Vote
Harris      54.862300
Other       45.576923
Stein       46.090909
Trump       56.124668
West        42.200000
Name: Age, dtype: float64
```

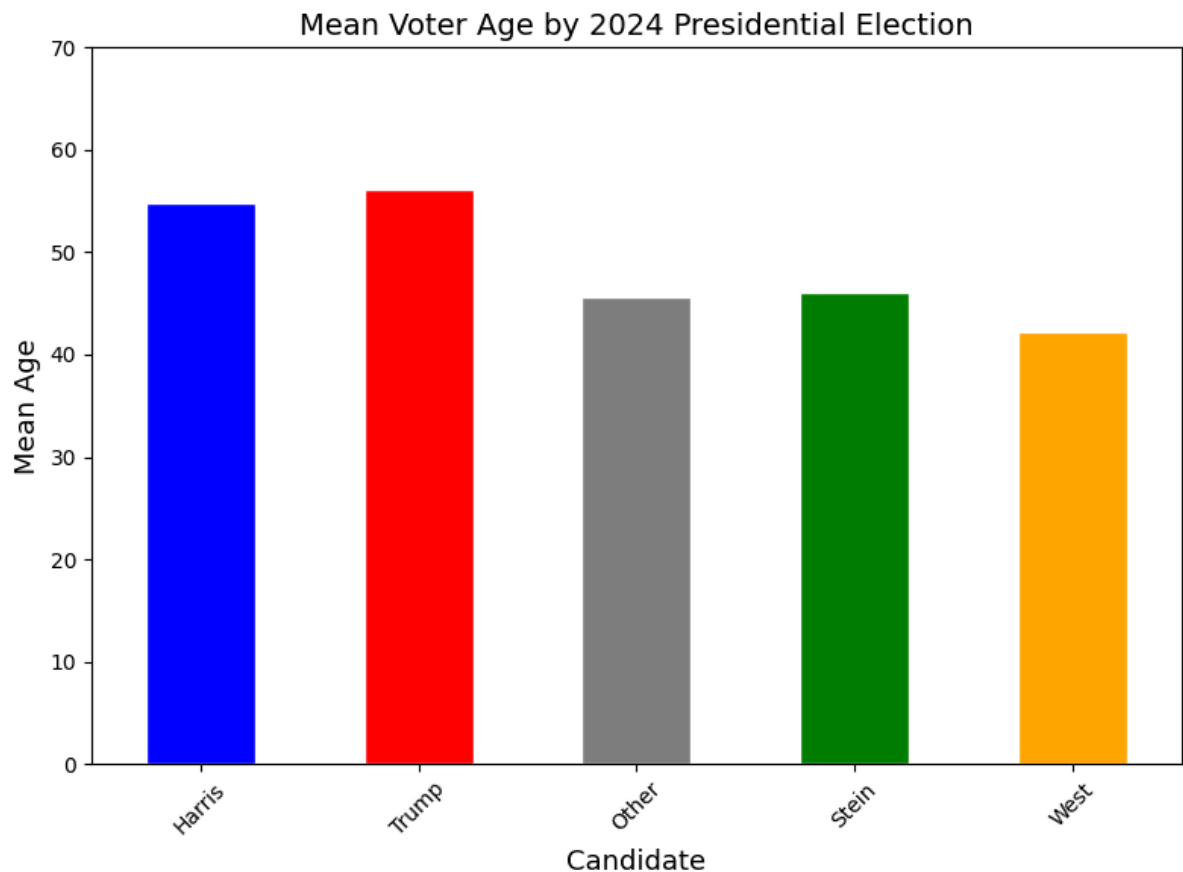
```
In [20]: # A bar chart to showcase the relationship between age and candiate selection

fig, ax = plt.subplots(figsize=(8, 6))

mean_ages = df_select.groupby("President Vote")["Age"].mean().reindex(["Harris", "Trump", "Other", "Stein", "West"])

mean_ages.plot.bar(color=["blue", "red", "gray", "green", "orange", "purple"],
edgecolor="white", ax=ax, width=0.5)

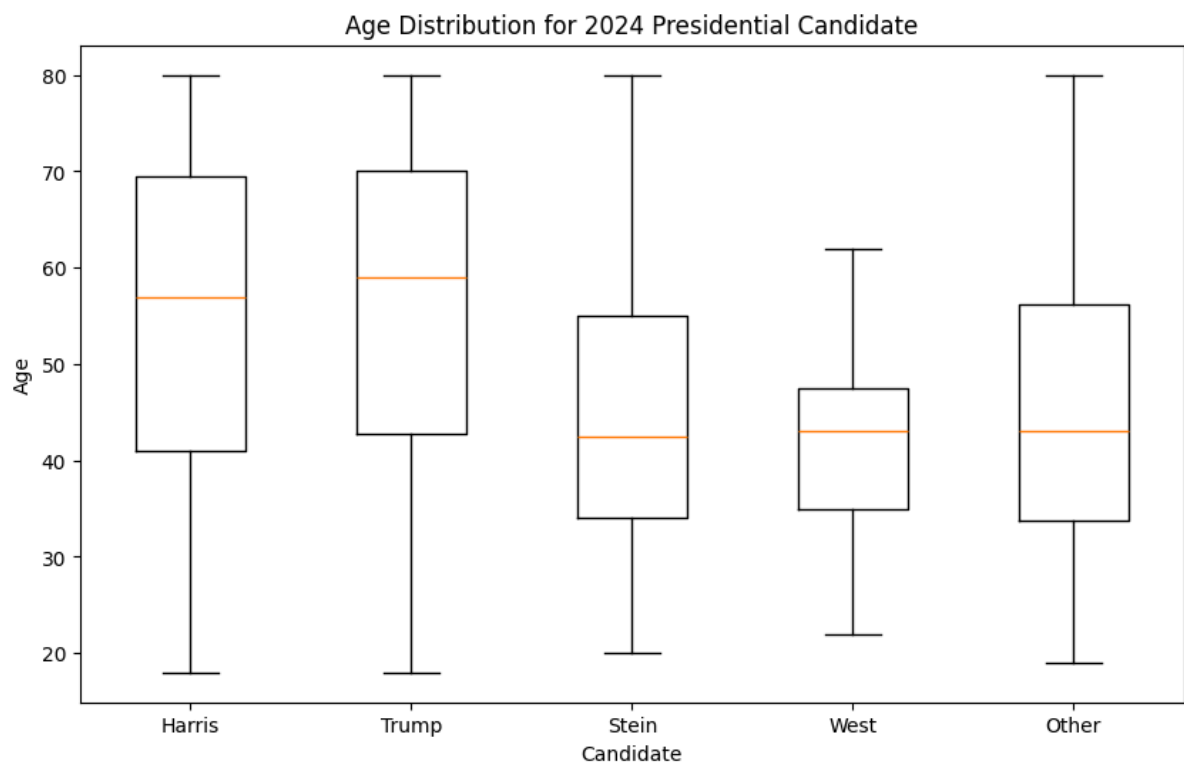
ax.set_xlabel("Candidate", fontsize=13)
ax.set_ylabel("Mean Age", fontsize=13)
ax.set_title("Mean Voter Age by 2024 Presidential Election", fontsize=14)
ax.set_ylim(0, 70)
ax.tick_params(axis='x', rotation=45)
plt.tight_layout()
plt.show()
```



Now to better see the distribution of the ages of each candidate the variance of the data we can use a side by side boxplot. This will allow us to exclude large or too small values which might skew the results


```
In [21]: # split the dataset for each candidate so we can make a box plot for each one,
# for a side by side comparison
df_harris = df_select[df_select["President Vote"] == "Harris"]
df_trump = df_select[df_select["President Vote"] == "Trump"]
df_stein = df_select[df_select["President Vote"] == "Stein"]
df_west = df_select[df_select["President Vote"] == "West"]
df_other = df_select[df_select["President Vote"] == "Other"]

plt.figure(figsize=(10, 6))
plt.boxplot([df_harris["Age"], df_trump["Age"], df_stein["Age"], df_west["Age"], df_other["Age"]],
            tick_labels=["Harris", "Trump", "Stein", "West", "Other"])
plt.ylabel("Age")
plt.xlabel("Candidate")
plt.title("Age Distribution for 2024 Presidential Candidate")
plt.show()
```



Now we will do Kruskal-Wallis Hypothesis Test

H₀ (null hypothesis): The age distributions of all candidates are the same, so there is no difference in the age of who voted for each candidate.

H₁ (alternative hypothesis): At least one candidate has a significantly different age distribution than the others.

```
In [22]: stat, p_value = sp.stats.kruskal(df_harris["Age"], df_trump["Age"], df_stein["Age"], df_west["Age"], df_other["Age"])
print(f"Kruskal-Wallis H statistic: {stat:.4f}")
print(f"P-value: {p_value:.2e}")
```

Kruskal-Wallis H statistic: 33.4709
P-value: 9.57e-07

```
In [23]: # we will use p = 0.05 to determine whether to reject or fail to reject the null
ll
if p_value < 0.05:
    print("Reject null hypothesis - At least one candidate has a significantly different age distribution than the others")
else:
    print("Fail to reject null - The age distributions of all candidates are the same")
```

Reject null hypothesis - At least one candidate has a significantly different age distribution than the others

From this data exploration, we can see several patterns in the relationship between respondent age and their candidate choice.

First the descriptive stats reveal that Trump voters have the highest mean age at 56.1 years, followed by Harris voters at 54.9 years. Third party candidates tend to attract younger voters, with West voters averaging 42.2 years and Stein voters at 46.1 years. The overall sample has a mean age of 55.2 years, suggesting the dataset skews older.

The boxplots further support this trend, showing that Trump voters have the highest median age, followed closely by Harris voters. Third party candidates attract noticeably younger voters with lower medians.

The Kruskal-Wallis test produced an H statistic of 33.47 and a p-value of 9.57e-07, which is far below the significance threshold of $\alpha = 0.05$, we had chosen. We therefore reject the null hypothesis, that the age distributions across candidate groups are statistically significantly different and did not occur by chance.

Some issues to note are that third party candidates such as Stein, West, and Other have considerably smaller sample sizes compared to Harris and Trump, meaning conclusions drawn about those groups may not be as strong. Additionally, age is likely not the only factor influencing vote choice, as possible confounders include race, income, and party identification.

Nevertheless, the statistically significant result confirms that age is a meaningful variable in understanding 2024 presidential vote choice.

Conclusion 3: Voter Identification versus Voter Turnout

```
In [24]: import pandas as pd
import scipy as sp
import numpy as np
from matplotlib import pyplot
```

```
In [25]: # Loading in the dataset

anes2024_csv = "../../../dataset/data_2024/anes_timeseries_2024_csv_20250808.csv"
df2024 = pd.read_csv(anes2024_csv)

df2024_select = df2024[["V241501x", "V241567x", "V241566x", "V241039", "V240108a",
                        "V240002c", "V242067"]]

df2024_select
```

C:\Users\james\AppData\Local\Temp\ipykernel_9764\1533670907.py:4: DtypeWarning: Columns (0: V240101a, 1: V240101c, 2: V240101d, 3: V240103a, 4: V240103c, 5: V240103d, 6: V240104a, 7: V240104c, 8: V240104d, 9: V240105a, 10: V240105c, 11: V240105d, 12: V240106a, 13: V240106c, 14: V240106d, 15: V240108a, 16: V240108c, 17: V240108d, 18: V243002, 19: V243003, 20: V243004, 21: V243005, 22: V243006, 23: V243009, 24: V243050, 25: V243051, 26: V243052, 27: V243053, 28: V243056, 29: V243057, 30: V243058) have mixed types. Specify dtype option on import or set low_memory=False.

```
df2024 = pd.read_csv(anes2024_csv)
```

Out[25]:

	V241501x	V241567x	V241566x	V241039	V240108a	V240002c	V242067
0	3	5	27	-1	0.809064	2	2
1	4	5	26	-1	2.575472	2	-1
2	1	5	24	-1	0.798864	2	1
3	4	3	12	-1	0.244757	2	-1
4	1	3	15	-1	0.239209	2	2
...	...	...	...	...	...	...	...
5516	-8	2	5	-1	0.19183	1	-6
5517	1	3	16	-1	2.869063	2	1
5518	2	3	12	-9	2.092296	1	-6
5519	2	4	19	-1	2.826738	2	1
5520	1	4	22	-1	1.309934	2	2

5521 rows × 7 columns

Important columns:

- V240001: 2024 Time Series Case ID
- V240002c: Pre & Post Interview Completion Status
- V240108b: Full Post-Election Survey Weights - Note: The ANES Guidebook instructs that post-survey weights be used whenever there is any post-survey data
- V241012: PRE: IS R REGISTERED TO VOTE
- V241221: PRE: PARTY ID: DOES R THINK OF SELF AS DEMOCRAT, REPUBLICAN, OR INDEPENDENT
- V242066: POST: DID R VOTE FOR PRESIDENT
- V242067: POST: FOR WHOM DID R VOTE FOR PRESIDENT

This section seeks to find patterns in the relationship between registered voters' self-identified political affiliation, and the level of voter turnout for each corresponding identity. The specific candidate for which the respondent voted will not be considered.

```
In [26]: # Load dataset
df2024e_select = df2024[["V240001", "V240002c", "V240108b", "V241012", "V241221", "V242066"]]
df2024e_select
```

Out[26]:

	V240001	V240002c	V240108b	V241012	V241221	V242066
0	140001	2	.780974621163524	1	2	1
1	140002	2	2.50707498962934	1	5	-1
2	140003	2	.814400897948447	1	1	1
3	140004	2	.263490124186044	1	2	-1
4	140005	2	.216673241876399	1	3	1
...	...	...	...	...	...	...
5516	399764	1		1	-8	-6
5517	399815	2	1.90730910362323	1	1	1
5518	399830	1		1	3	-6
5519	399841	2	2.56436072555069	1	1	1
5520	399909	2	1.34915930327876	1	2	1

5521 rows × 6 columns

```
In [27]: # Dictionaries for conversion

# V240001 will not be converted as it is the entry IDs

# V241221 - Identified Political Party
party_dict = {
    -9: "Refused",
    -8: "Don't Know",
    -1: "Inapplicable",
    0: "No Preference",
    1: "Democrat",
    2: "Republican",
    4: "Independent",
    5: "Other"
}

#V241012 - Was R registered
registered_dict = {
    -9: "Refused",
    -8: "Don't Know",
    -1: "Inapplicable",
    1: "Yes",
    2: "No",
}

#V242066 - Did R vote for President
voted_dict = {
    -7: "Insufficient Partial",
    -6: "No Post Interview",
    -1: "Inapplicable",
    1: "Yes",
    2: "No"
}

# V240002c (survey completion) already covered above as complete_survey_dict
```

```

In [28]: # Making the dataset more readable
df2024e_select["ID"] = df2024e_select["V240001"]
df2024e_select["Registration"] = df2024e_select["V241012"].map(registered_dict)
df2024e_select["Party ID"] = df2024e_select["V241221"].map(party_dict)
df2024e_select["Voted for President"] = df2024e_select["V242066"].map(voted_dict)
df2024e_select["Completed Survey"] = df2024e_select["V240002c"].map(complete_survey_dict)

# Clean up 'Weights': replace empty strings with 0.
df2024e_select["Weights"] = df2024e_select["V240108b"]
df2024e_select["Weights"] = df2024e_select["Weights"].apply(lambda x: 0 if x == '' or x == '' else x)
df2024e_select["Weights"] = df2024e_select["Weights"].astype(float)

df2024e_clean = df2024e_select[["ID", "Completed Survey", "Registration", "Party ID",
                                "Voted for President", "Weights"]]

#Remove entries with NaN
df2024e_clean = df2024e_clean.dropna()

df2024e_clean

```

Out[28]:

	ID	Completed Survey	Registration	Party ID	Voted for President	Weights
0	140001	1	Yes	Republican	Yes	0.780975
1	140002	1	Yes	Other	Inapplicable	2.507075
2	140003	1	Yes	Democrat	Yes	0.814401
3	140004	1	Yes	Republican	Inapplicable	0.263490
7	140008	1	No	Republican	Inapplicable	0.618924
...	...	...	...	...	...	...
5515	399579	1	Yes	Other	Yes	0.094869
5516	399764	0	Yes	Don't Know	No Post Interview	0.000000
5517	399815	1	Yes	Democrat	Yes	1.907309
5519	399841	1	Yes	Democrat	Yes	2.564361
5520	399909	1	Yes	Republican	Yes	1.349159

3968 rows × 6 columns

We want to select only respondents that had completed both surveys, and were registered to vote, with a positive "Yes" response.

```
In [29]: # Select only entries that completed both pre- and post- survey
df2024e_relevant = df2024e_clean[df2024e_clean["Completed Survey"] == 1]

# Drop No, Inapplicable, Don't Know Entries for voting Registration - we only
want registered voters
df2024e_relevant = df2024e_relevant[~df2024e_relevant["Registration"].isin(['N
o', 'Don\'t Know', 'Inapplicable'])]

df2024e_relevant
```

Out[29]:

	ID	Completed Survey	Registration	Party ID	Voted for President	Weights
0	140001	1	Yes	Republican	Yes	0.780975
1	140002	1	Yes	Other	Inapplicable	2.507075
2	140003	1	Yes	Democrat	Yes	0.814401
3	140004	1	Yes	Republican	Inapplicable	0.263490
9	140011	1	Yes	Democrat	Inapplicable	1.030726
...	...	...	...	...	...	...
5510	399081	1	Yes	Democrat	Inapplicable	1.953840
5515	399579	1	Yes	Other	Yes	0.094869
5517	399815	1	Yes	Democrat	Yes	1.907309
5519	399841	1	Yes	Democrat	Yes	2.564361
5520	399909	1	Yes	Republican	Yes	1.349159

3171 rows × 6 columns

For the purposes of this data exploration, an "Inapplicable" response to whether the respondent voted or not will be treated as a "No".

```

In [30]: # Condense No, Inapplicable into No for Presidential Voting
def condense_negative(response):
    if response == "Inapplicable":
        return "No"
    else:
        return response

df2024e_relevant["Voted for President"] = df2024e_relevant["Voted for Presiden
t"].apply(condense_negative)

df2024e_relevant

```

Out[30]:

	ID	Completed Survey	Registration	Party ID	Voted for President	Weights
0	140001	1	Yes	Republican	Yes	0.780975
1	140002	1	Yes	Other	No	2.507075
2	140003	1	Yes	Democrat	Yes	0.814401
3	140004	1	Yes	Republican	No	0.263490
9	140011	1	Yes	Democrat	No	1.030726
...	...	...	...	...	...	...
5510	399081	1	Yes	Democrat	No	1.953840
5515	399579	1	Yes	Other	Yes	0.094869
5517	399815	1	Yes	Democrat	Yes	1.907309
5519	399841	1	Yes	Democrat	Yes	2.564361
5520	399909	1	Yes	Republican	Yes	1.349159

3171 rows × 6 columns


```

In [31]: # Creating a crosstab summarization of voter turnout by party ID
voter_turnout_tab = pd.crosstab(index=df2024e_relevant['Party ID'],
                                columns=df2024e_relevant['Voted for President'],
                                values=df2024e_relevant['Weights'],
                                aggfunc="sum"
                                )

voter_turnout_tab = voter_turnout_tab.reindex(index=['Democrat', 'Republican',
'No Preference', 'Don\'t Know', 'Refused', 'Other'],
                                              columns=['Yes', 'No'])

# Simple sum of weighted values, shows relative scale of Party ID categories
print(voter_turnout_tab)

print("\n")

# Calculating ratios for each category
voter_turnout_tab_percent = pd.crosstab(index=df2024e_relevant['Party ID'],
                                          columns=df2024e_relevant['Voted for President'],
                                          values=df2024e_relevant['Weights'],
                                          aggfunc="sum",
                                          normalize="index"
                                          )

voter_turnout_tab_percent = voter_turnout_tab_percent.reindex(index=['Democrat', 'Republican', 'No Preference', 'Don\'t Know', 'Refused', 'Other'],
                                                              columns=['Yes', 'No'])

voter_turnout_tab_percent = voter_turnout_tab_percent.round(4) * 100

print(voter_turnout_tab_percent)

```

Voted for President	Yes	No
Party ID		
Democrat	1318.877991	261.279551
Republican	1075.981712	240.182408
No Preference	11.263082	9.884599
Don't Know	1.054259	2.785557
Refused	7.434706	3.391035
Other	92.524240	29.443696

Voted for President	Yes	No
Party ID		
Democrat	83.46	16.54
Republican	81.75	18.25
No Preference	53.26	46.74
Don't Know	27.46	72.54
Refused	68.68	31.32
Other	75.86	24.14

```
In [32]: # Calculating ratio of Party Identifications to Overall Weighted Sum
party_ratio_tab = pd.crosstab(index=df2024e_relevant['Party ID'],
                             columns=df2024e_relevant['Registration'],
                             values=df2024e_relevant['Weights'],
                             aggfunc="sum",
                             normalize="all")
party_ratio_tab = party_ratio_tab.reindex(index=['Democrat', 'No Preference',
'Republican', 'Don\'t Know', 'Other', 'Refused'],
                                          columns=['Yes'])

print(party_ratio_tab)

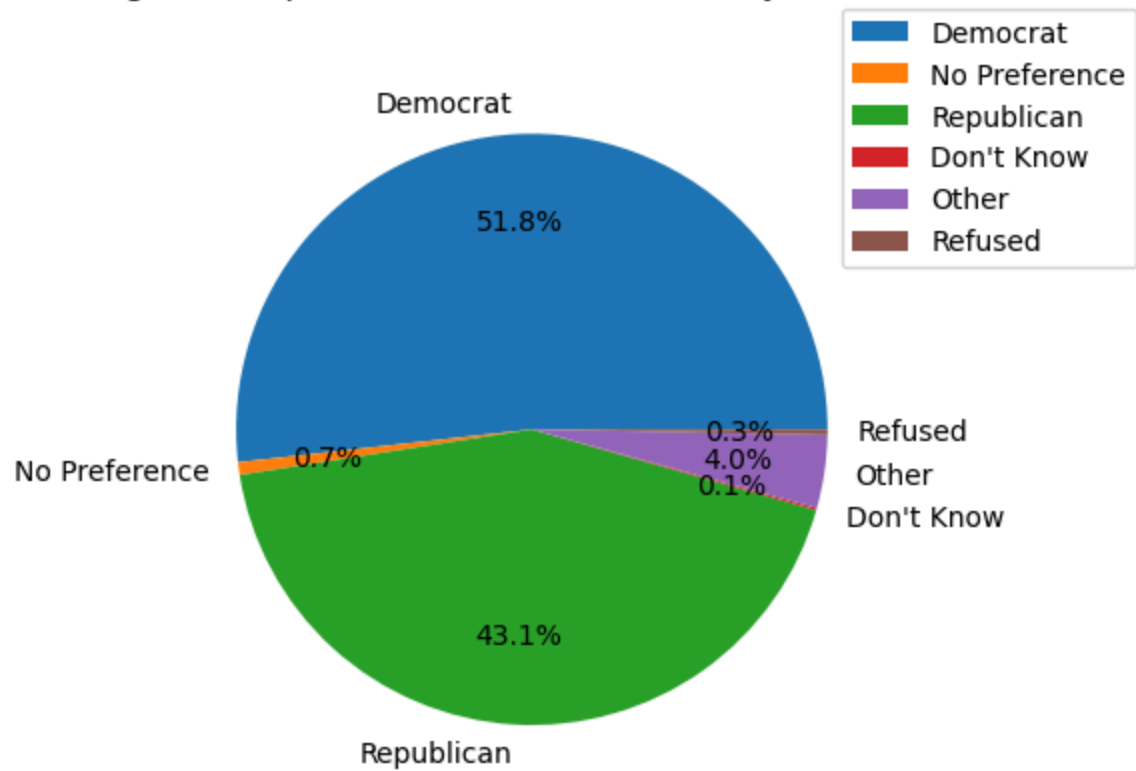
party_ratio_pieplot = party_ratio_tab.plot.pie(
    subplots=True,
    title="Weighted Proportion of Self-Identified Party Affiliations",
    labeldistance=1.1,
    autopct='%1.1f%%',
    pctdistance=.7
)

pyplot.legend(loc='best', bbox_to_anchor=(0.9, 0.7))
```

Registration	Yes
Party ID	
Democrat	0.517388
No Preference	0.006924
Republican	0.430950
Don't Know	0.001257
Other	0.039936
Refused	0.002837

Out[32]: <matplotlib.legend.Legend at 0x245d770e990>

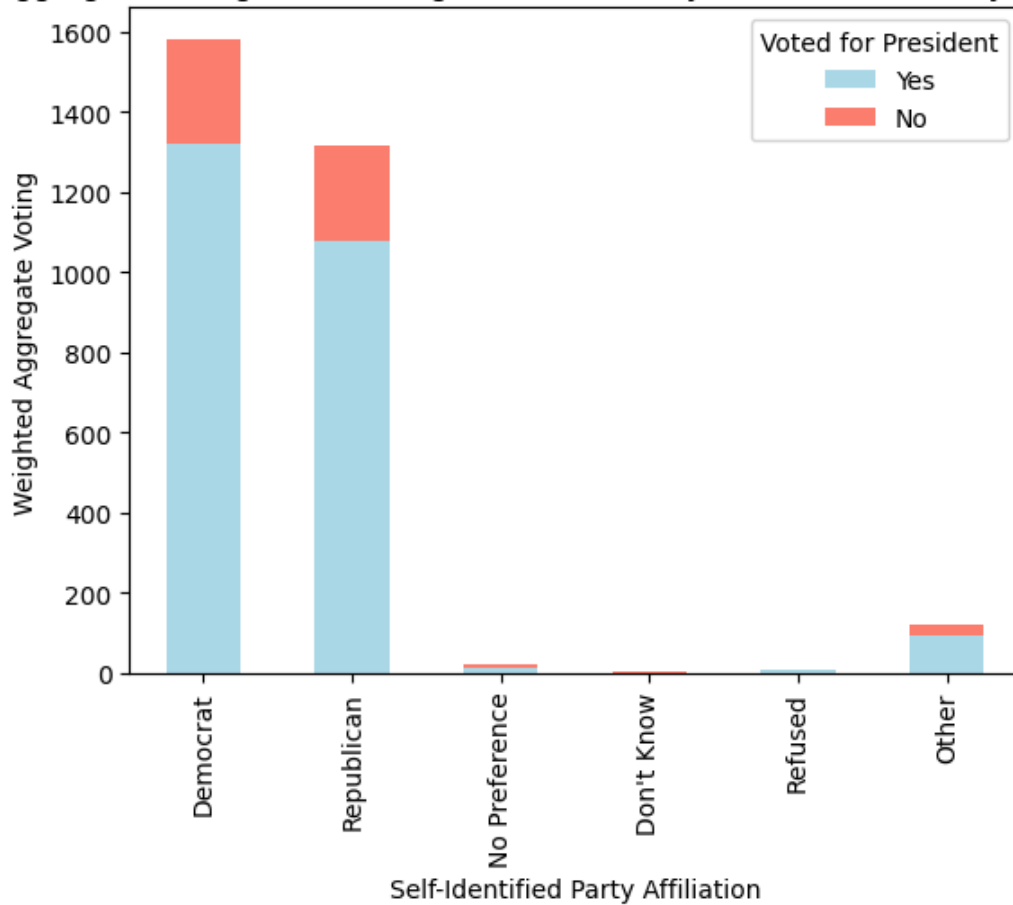
Weighted Proportion of Self-Identified Party Affiliations



```
In [33]: voter_turnout_barplot = voter_turnout_tab.plot.bar(  
        stacked=True,  
        title="Aggregate Voting Ratio Of Registered Voters By Self-Identified Party  
        Affiliation",  
        color=['lightblue', 'salmon']  
    )  
  
voter_turnout_barplot.set_xlabel("Self-Identified Party Affiliation")  
voter_turnout_barplot.set_ylabel("Weighted Aggregate Voting")
```

Out[33]: Text(0, 0.5, 'Weighted Aggregate Voting')

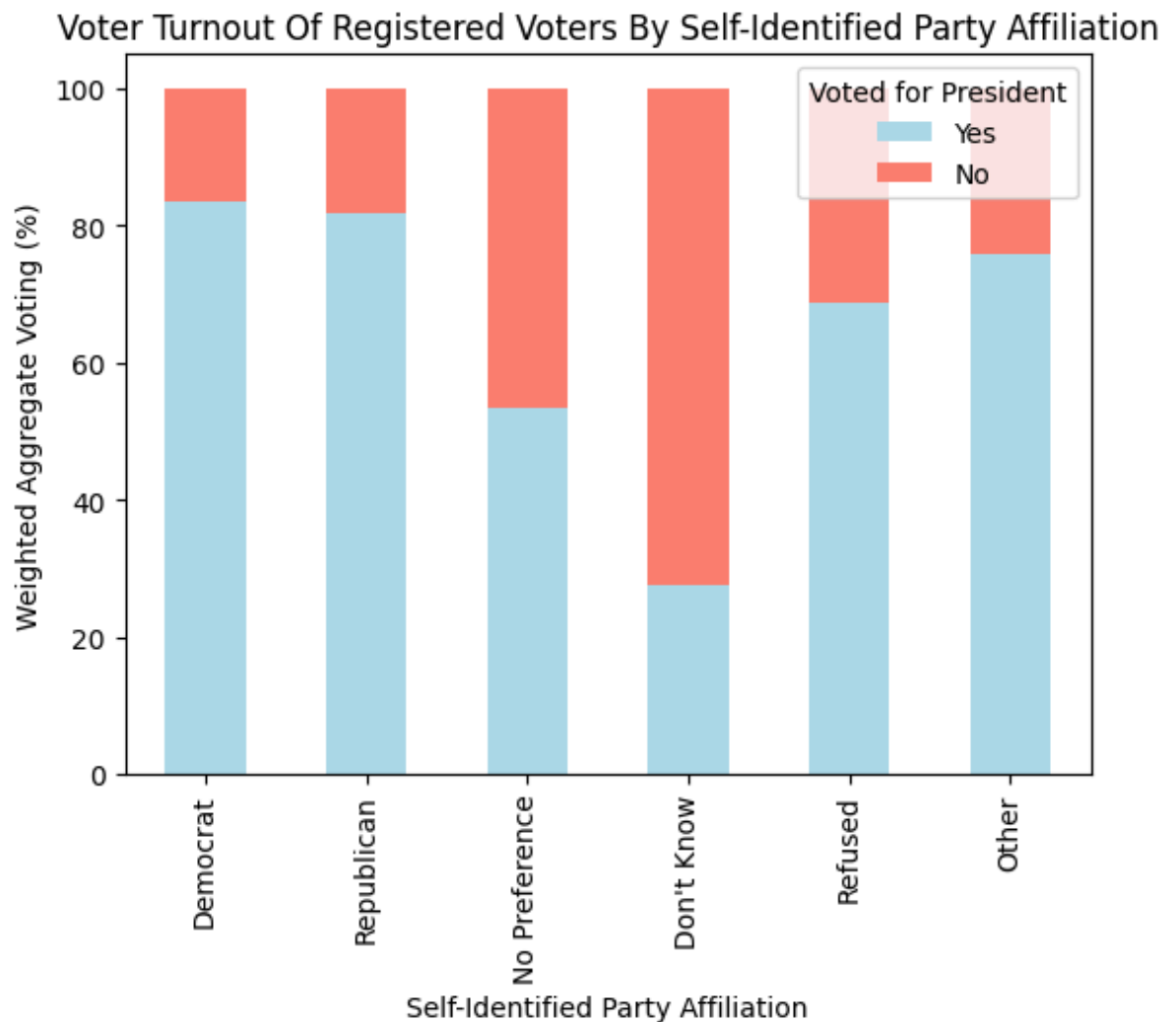
Aggregate Voting Ratio Of Registered Voters By Self-Identified Party Affiliation



```
In [34]: voter_turnout_percent_barplot = voter_turnout_tab_percent.plot.bar(
        stacked=True,
        title="Voter Turnout Of Registered Voters By Self-Identified Party Affilia
        tion",
        color=['lightblue', 'salmon']
    )

voter_turnout_percent_barplot.set_xlabel("Self-Identified Party Affiliation")
voter_turnout_percent_barplot.set_ylabel("Weighted Aggregate Voting (%)")
```

Out[34]: Text(0, 0.5, 'Weighted Aggregate Voting (%)')



From this preliminary data exploration, we can see that there are several patterns within the data in relation to political party affiliation and voter turnout.

First, self-identified Democrats and Republicans have a relatively similar (weighted) voter turnout, with 83% and 81% of registered voters for each showing up to vote for President. They also make up the vast majority of registered respondents, at 52% and 43% respectively (see pie chart).

The 3rd-highest turnout rate at 76%, and 3rd-largest group, self-identified as Other, which was an open response answer. This was followed by those who Refused to answer at 69%, and those with No Preference at 53%. Finally, those who selected "Don't Know" had the lowest level of voter turnout, at just 27%.

These trends can be found to generally correspond with varying level of interest and involvement in the political process. Self-identified Democrats and Republicans were most invested in the Presidential race, while many registered voters without enough knowledge of or interest in politics to form an opinion did not end up voting.

We will test these conclusions with a hypothesis test:

Null Hypothesis: there is no difference between voter turnout between the different voter identification groups, and these observed results are due to random chance.

Alternative Hypothesis: there is a real difference in voter turnout between these different groups.

```
In [35]: #Chi^2 Test for Independence with the weighted cumulative contingency table
turnout_chi2 = sp.stats.chi2_contingency(voter_turnout_tab)

print("P-value: ", turnout_chi2.pvalue)
```

P-value: 7.3551633845472e-05

From this Chi-squared test, we get a P-value which is well below the statistical threshold of 0.05. Therefore, we reject the null hypothesis and there is sufficient evidence to support the alternative conclusion that there is a statistically significant relationship between voters' party affiliation (or lack thereof) and their turnout.

There are a few limitations to the conclusions drawn from this data. Only respondents who completed both the pre- and post- survey and affirmed that they were registered to vote in the pre-survey were considered. The specific candidate respondents voted for was not considered, which may or may not be different from their self-identified party affiliation. Additionally, the number of respondents for the "No Preference", "Don't Know", and "Refused" categories are quite few, meaning that the conclusions drawn from them may not be as strong.

However, the characteristics of the dataset explored in this section still hold much value for considering the relationship between voter self-identification and their subsequent turnout at the election.

Conclusion 4: American Identity versus Presidential Candidate Perceptions

```
In [36]: import pandas as pd
import scipy as sp
import numpy as np
import matplotlib.pyplot as plt
from wcorr import WeightedCorr
```

This file aims to analyze two separate relationships between the columns V242536 (American Identity) and V241156 (Feeling Thermometer: Kamala Harris), and columns V242536 (American Identity) and V241157 (Feeling Thermometer: Donald Trump). These two relationships primarily seek to analyze the correlation and impact of nationalism and patriotism in an individual to their preferential candidate.

Because the column associated with American Identity was conducted as a post election survey, the column V240002c (Pre/Post Interview Completion Status) is also utilized to specifically filter for interviewees who filled out the post election portion of the survey. In addition, the column V240108b, which contains the sampling weights for each row, is also important for the purposes of this analysis when performing the statistical tests.

```
In [37]: # Loading in the dataset and associated columns
anes2024_csv = "../../dataset/data_2024/anes_timeseries_2024_csv_20250808.csv"
df = pd.read_csv(anes2024_csv)

df_filtered = df[["V240002c", "V242536", "V241156", "V241157", "V240108b"]]

df_filtered
```

C:\Users\james\AppData\Local\Temp\ipykernel_9764\2275628996.py:3: DtypeWarning: Columns (0: V240101a, 1: V240101c, 2: V240101d, 3: V240103a, 4: V240103c, 5: V240103d, 6: V240104a, 7: V240104c, 8: V240104d, 9: V240105a, 10: V240105c, 11: V240105d, 12: V240106a, 13: V240106c, 14: V240106d, 15: V240108a, 16: V240108c, 17: V240108d, 18: V243002, 19: V243003, 20: V243004, 21: V243005, 22: V243006, 23: V243009, 24: V243050, 25: V243051, 26: V243052, 27: V243053, 28: V243056, 29: V243057, 30: V243058) have mixed types. Specify dtype option on import or set low_memory=False.

```
df = pd.read_csv(anes2024_csv)
```

Out[37]:

	V240002c	V242536	V241156	V241157	V240108b
0	2	1	0	100	.780974621163524
1	2	3	50	50	2.50707498962934
2	2	5	90	0	.814400897948447
3	2	2	50	70	.263490124186044
4	2	4	5	60	.216673241876399
...	...	...	...	...	...
5516	1	-6	50	50	
5517	2	3	85	0	1.90730910362323
5518	1	-6	50	50	
5519	2	3	90	0	2.56436072555069
5520	2	4	15	70	1.34915930327876

5521 rows × 5 columns


```

In [38]: # Improving readability of the filtered dataframe by renaming the columns first
df_filtered = df_filtered.rename(columns={'V240002c': 'Completed Post Survey',
'V242536': 'American Identity',
'V241156': 'Harris Feeling Thermometer',
'V241157': 'Trump Feeling Thermometer', 'V240108b': 'Weights'})
df_filtered

# Filtering rows of the dataframe so that only those who completed the post el
ection survey are considered
df_filtered = df_filtered[df_filtered['Completed Post Survey'] == 2]
df_filtered

# Filtering rows of the dataframe with invalid American Identity values are re
moved
# The guide states that any negative value indicates the interviewee did not c
omplete this question for a variety of reasons,
# thus those specific rows will not be considered.
df_filtered = df_filtered[df_filtered['American Identity'].isin([1, 2, 3, 4,
5])]
df_filtered

# Continuing the filter through the feeling thermometers, removing any invalid
rows
# Similarly to the previous filtering, the guide states that any negative valu
es represent unusable data and will thus be removed.
df_filtered = df_filtered[(df_filtered['Harris Feeling Thermometer'] >= 0) &
(df_filtered['Trump Feeling Thermometer'] >= 0)]
df_filtered

# We can observe through the guide that the american identity values are actua
lly originally in reverse order, with 1 being the strongest
# identity and 5 being the weakest. For the sake of consistency for the later
plots and statistical testing, it will be corrected here so that
# the higher the identity value, the more strongly the interviewee feels about
their american identity.
df_filtered['American Identity'] = 6 - df_filtered['American Identity']
df_filtered

# Filling in empty weight values with 0 weight
df_filtered["Weights"] = df_filtered["Weights"].apply(lambda x: 0 if x == ' '
or x == '' else x)
df_filtered["Weights"] = df_filtered["Weights"].astype(float)
df_filtered

# Creating two separate dataframes for both Harris and Trump
df_harris = df_filtered[['American Identity', 'Harris Feeling Thermometer', 'W
eights']]
df_trump = df_filtered[['American Identity', 'Trump Feeling Thermometer', 'Wei
ghts']]
display(df_harris)
display(df_trump)

```

American Identity	Harris Feeling Thermometer	Weights
0	5	0 0.780975
1	3	50 2.507075
2	1	90 0.814401
3	4	50 0.263490
4	2	5 0.216673
...	...	...
5514	4	48 0.596674
5515	5	100 0.094869
5517	3	85 1.907309
5519	3	90 2.564361
5520	2	15 1.349159

4652 rows × 3 columns

American Identity	Trump Feeling Thermometer	Weights
0	5	100 0.780975
1	3	50 2.507075
2	1	0 0.814401
3	4	70 0.263490
4	2	60 0.216673
...	...	...
5514	4	48 0.596674
5515	5	0 0.094869
5517	3	0 1.907309
5519	3	0 2.564361
5520	2	70 1.349159

4652 rows × 3 columns

Now that we have cleaned our dataset, we can move to plot our data in order to draw some initial conclusions and observations about the relationship between American Identity and an individual's preference for Harris/Trump as presidential candidates.

We will use boxplots to graph the distributions of the feeling thermometer scores in accordance to the american identity of the interviewees.

```

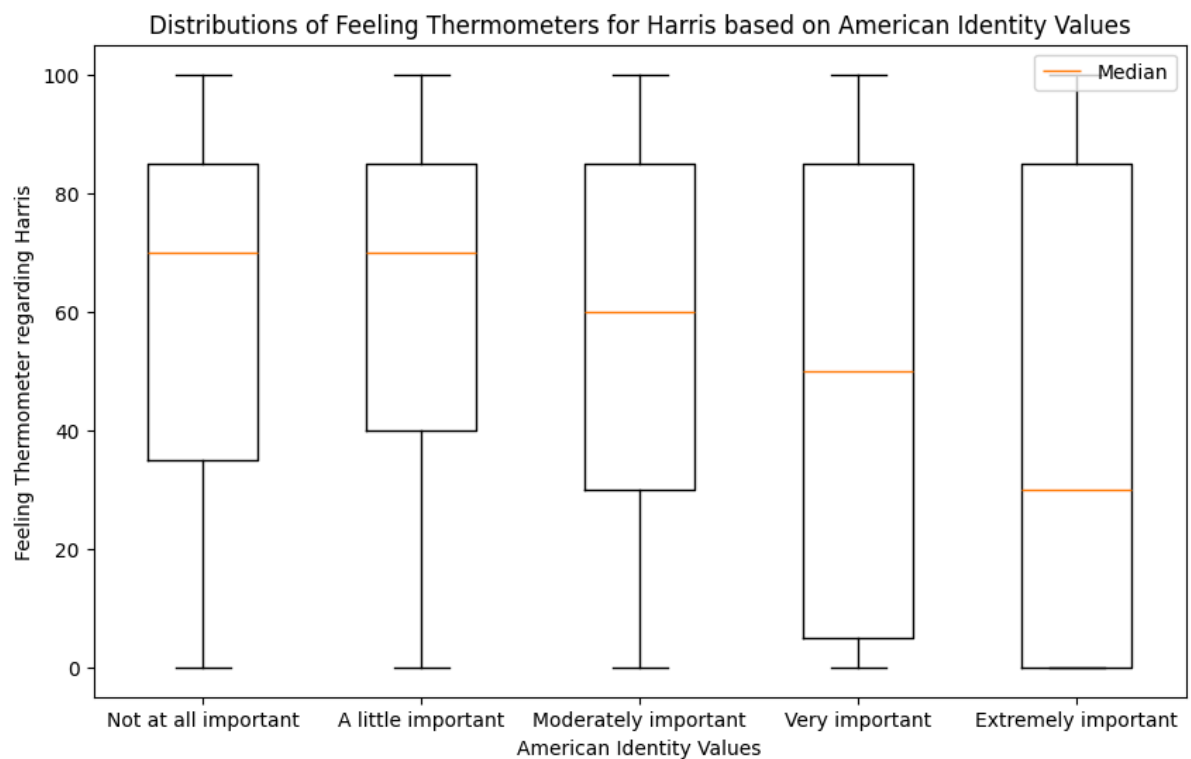
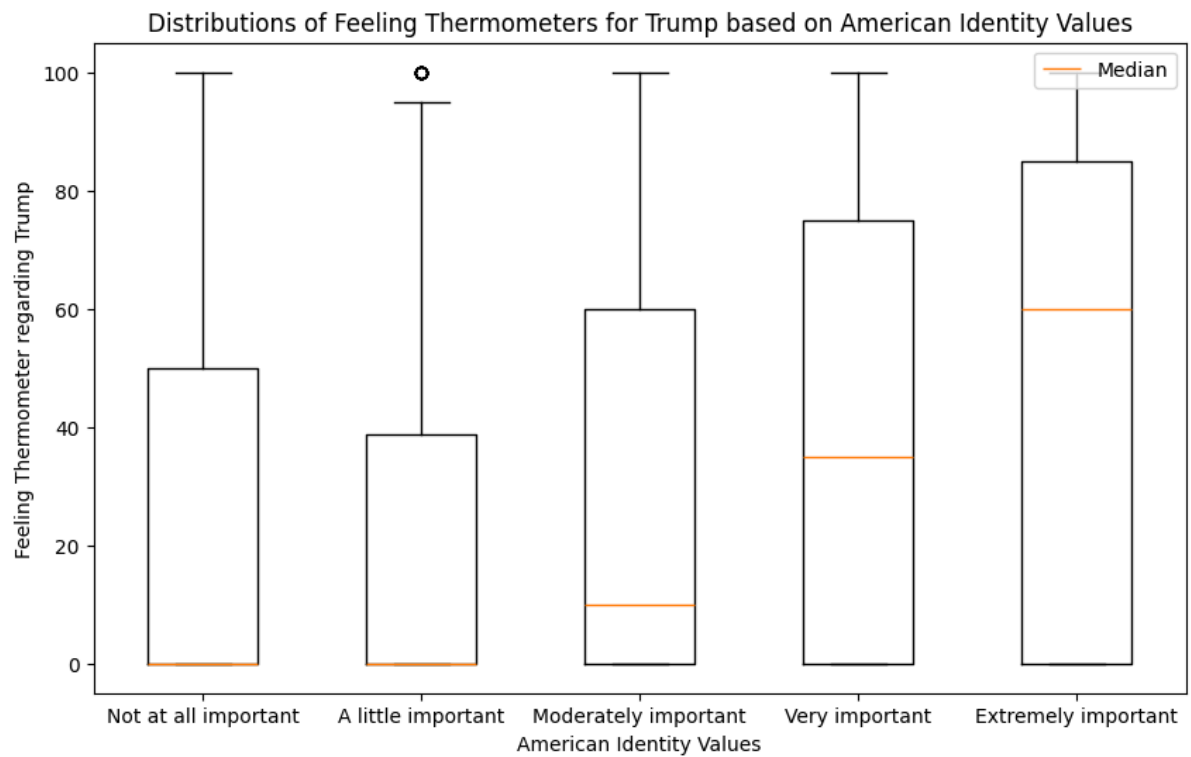
In [39]: # Split apart the two dataframes even further so that we can map the distributions between identity scores
df_trump_1 = df_trump[df_trump['American Identity'] == 1]
df_trump_2 = df_trump[df_trump['American Identity'] == 2]
df_trump_3 = df_trump[df_trump['American Identity'] == 3]
df_trump_4 = df_trump[df_trump['American Identity'] == 4]
df_trump_5 = df_trump[df_trump['American Identity'] == 5]

df_harris_1 = df_harris[df_harris['American Identity'] == 1]
df_harris_2 = df_harris[df_harris['American Identity'] == 2]
df_harris_3 = df_harris[df_harris['American Identity'] == 3]
df_harris_4 = df_harris[df_harris['American Identity'] == 4]
df_harris_5 = df_harris[df_harris['American Identity'] == 5]

# Plot the newly formed dataframes into separate boxplots
plt.figure(figsize=(10, 6))
plt.boxplot([df_trump_1['Trump Feeling Thermometer'], df_trump_2['Trump Feeling Thermometer'], df_trump_3['Trump Feeling Thermometer'], df_trump_4['Trump Feeling Thermometer'], df_trump_5['Trump Feeling Thermometer']],
            tick_labels=['Not at all important', 'A little important', 'Moderately important', 'Very important', 'Extremely important'],
            label="Median")
plt.legend(loc='upper right')
plt.ylabel('Feeling Thermometer regarding Trump')
plt.xlabel('American Identity Values')
plt.title('Distributions of Feeling Thermometers for Trump based on American Identity Values')
plt.show()

plt.figure(figsize=(10, 6))
plt.boxplot([df_harris_1['Harris Feeling Thermometer'], df_harris_2['Harris Feeling Thermometer'], df_harris_3['Harris Feeling Thermometer'], df_harris_4['Harris Feeling Thermometer'], df_harris_5['Harris Feeling Thermometer']],
            tick_labels=['Not at all important', 'A little important', 'Moderately important', 'Very important', 'Extremely important'],
            label='Median')
plt.legend(loc='upper right')
plt.ylabel('Feeling Thermometer regarding Harris')
plt.xlabel('American Identity Values')
plt.title('Distributions of Feeling Thermometers for Harris based on American Identity Values')
plt.show()

```



In addition to these graphs, we will now perform a statistical test on the corresponding dataframes before coming to a more concrete conclusion on the relationship between an individual's American Identity and their preference for both Harris and Trump as presidential candidates.

I have chosen to use a Spearman's Correlation Coefficient test, since we have ordinal data, that being the ranking of american identity on a scale from 1 to 5, as well as a continuous variable of the feeling thermometers for both Harris and Trump. Additionally, the Spearman's Correlation Coefficient test does not depend on normality, and normality can not be assumed when it comes to the provided data for the feeling thermometers.

Hypothesis Tests:

H0 = The American Identity of an individual has no effect on their feeling thermometer score for Trump/Harris

HA = The American Identity of an individual has an effect on their feeling thermometer score for Trump/Harris

```
In [40]: # Performing the weighted spearman's correlation coefficient tests
trump_coef = WeightedCorr(xyw=df_trump[['American Identity', 'Trump Feeling Thermometer', 'Weights']]) (method='spearman')
harris_coef = WeightedCorr(xyw=df_harris[['American Identity', 'Harris Feeling Thermometer', 'Weights']]) (method='spearman')

trump_unweight = sp.stats.spearmanr(df_trump['American Identity'], df_trump['Trump Feeling Thermometer'])
harris_unweight = sp.stats.spearmanr(df_harris['American Identity'], df_harris['Harris Feeling Thermometer'])

display(trump_coef)
display(harris_coef)

display(trump_unweight.pvalue)
display(harris_unweight.pvalue)

np.float64(0.2918632889626895)

np.float64(-0.20401859865017727)

np.float64(3.727057919997921e-86)

np.float64(3.598337401231189e-43)
```

There are many observations that can be noted from both the statistical test and graphical plot that was created earlier regarding the relationship between american identity and the feeling thermometers of Trump and Harris.

Firstly, we can observe that our two p-values are incredibly small, much smaller than the accepted value of $\alpha = 0.05$, we can thus reject our null hypotheses and state that the american identity of an individual has a statistical significance on their feeling thermometer scores of Trump and Harris.

Additionally, it can be observed that there is a general positive trend in regards to an individual's american identity and their opinion of Trump. As noted by both the box plots and the spearman coefficient being greater than 0, the more an individual considers their american identity to be important to them, the higher they regard Trump as a political candidate. Inversely, a generally negative trend can be noted with american identity and the feeling thermometer score of Harris as a political candidate from the boxplots and correlation coefficient. This is seen with the median of the feeling thermometer generally decreasing as american identity grows in importance between the boxplots, as well as the corresponding spearman coefficient being less than 0.

There are still some issues that need to be discussed from these initial tests, namely the overall strength of the relationship presented for both Trump and Harris. As noted by the correlation coefficient especially, the values 0.29 and -0.20 suggest that the relationship between american identity and the feeling thermometer scores is not necessarily very strong, and in general can be stated to be moderate to weak. This suggests that while a relationship certainly does seem to exist, it might not be as prevalent as one may think and thus should be considered when performing analysis on the two variables.

These limitations can also be noticed in the boxplots as well. In every single boxplot in both graphs, one can note the clear maximum and minimum values still being from 100 to 0 respectively, thus implying that there still exist non outlier individuals in all the groups who vote out of the norm of the median. Additionally, there is a noticeable "floor" effect with the Trump boxplots and a "ceiling" effect with the Harris ones, that is to say that the lower and upper quartile respectively do not seem to change as the american identity becomes increasingly important. This supports the correlation coefficients being on the weaker side, since there are still those who vote weakly/strongly about the candidate across all the groups, thus decreasing the overall strength of the relationship between the two variables.

That being said, the indication that a relationship does exist means that these two variables are still worth analyzing and looking into for any future analysis that may come from this project.